Chapter 7:

Recurrent Neural Networks (RNNs)

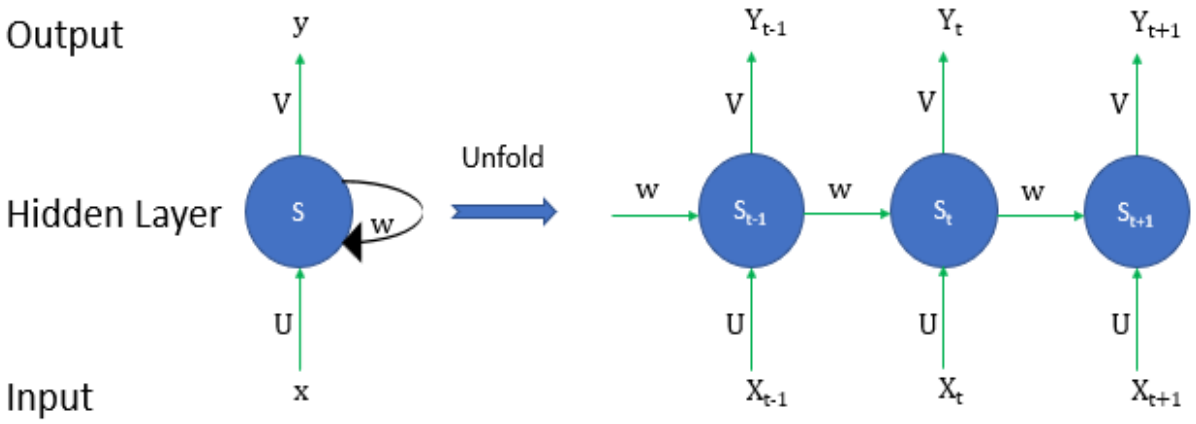
Recurrent Neural Networks (RNNs)

- specifically designed to handle sequential data, such as time-series data, text sequences, and other forms of sequence-based information.
- RNNs are equipped with feedback loops that enable them to maintain a memory of past inputs.
- This capability allows RNNs to capture temporal relationships and dependencies within sequential data

Key Characteristics:

- **Sequential Data Handling**: Processes input sequences one element at a time.
- **Memory Retention**: Maintains an internal state to store past information.
- **Shared Parameters**: Reduces model complexity by sharing weights across time steps.

Mathematical Formulation



- x_t: Input vector at time step t
- h_t: Hidden state at time step t
- h_{t-1}: Hidden state from the previous time step
- W_x: Weight matrix for the input-to-hidden connection
- W_h : Weight matrix for the hidden-to-hidden connection
- b_h: Bias vector for the hidden layer
- f: Activation function (e.g., tanh or ReLU)
- y_t: Output vector at time step t
- W_y: Weight matrix for the hidden-to-output connection
- b_y: Bias vector for the output layer
- g: Output activation function (optional)

Hidden State Update:

$$h_t=f(W_x x_t+W_h h_{t-1}+b_h)$$

Output Generation (optional, depending on the RNN variant):

$$y_t=g(W_y h_t+b_y)$$

Challenges with Basic RNNs

Vanishing Gradient Problem: Gradients become too small to influence learning over long sequences.

Exploding Gradient Problem: Gradients grow exponentially, leading to instability.

Variants of Recurrent Neural Networks:

Long Short-Term Memory (LSTM):

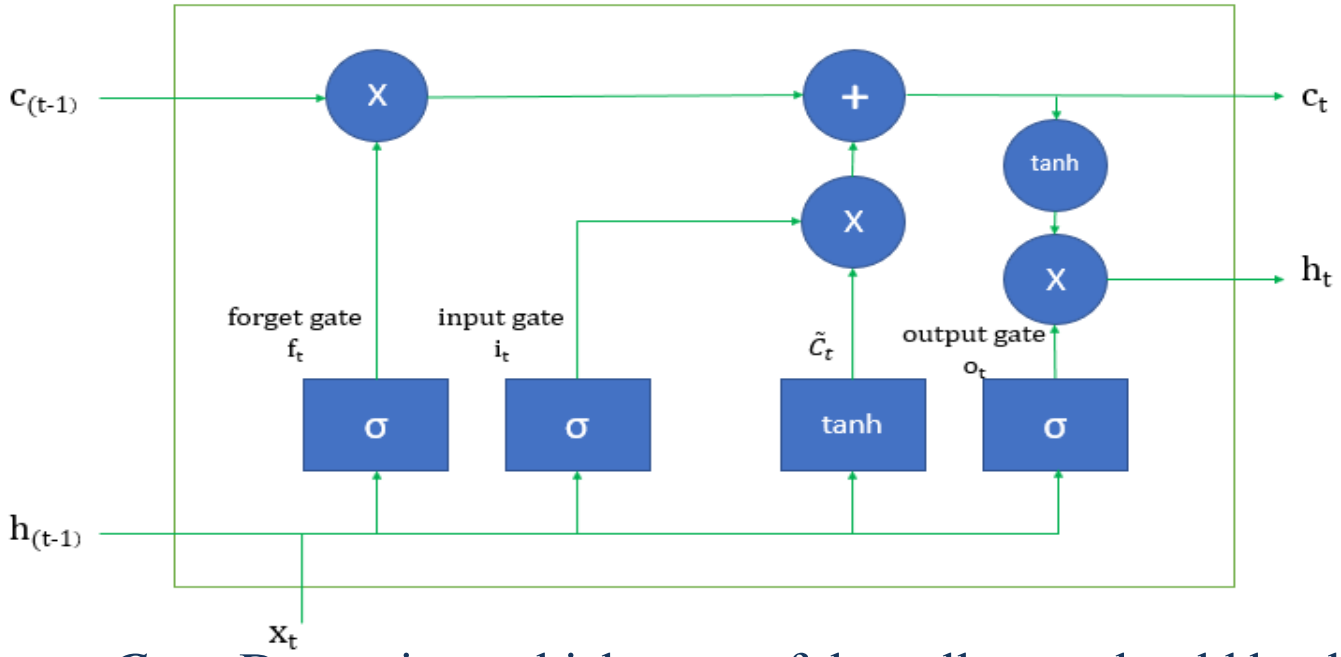
LSTM networks are a type of RNN architecture introduced to address the vanishing gradient problem, which hinders the training of basic RNNs over long sequences.

By introducing gates and a cell state, LSTMs can effectively retain important information over time.

Key Characteristics

- **Memory Cells**: Store information selectively over time.
- **Gating Mechanism**: Controls what information to keep, forget, and output.
- **Long-Term Dependency Handling**: Efficiently learns relationships over long sequences.

Long Short-Term Memory (LSTM):



- ***Forget Gate***: Determines which parts of the cell state should be discarded.
  - Forget gate output.
  - Weight matrix for forget gate.
  - Bias term.
  - Uses a sigmoid activation function to produce values between 0 (completely forget) and 1 (completely keep).
  - Formula:

    $$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Long Short-Term Memory (LSTM):

- ***Input* Gate**: Decides what new information to add to the cell state.
  - Input gate output.
  - Candidate values for the cell state.
  - Combines a sigmoid gate (for importance) with a tanh layer (to generate new candidate values).
  - Formula:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(w_c . [h_{t-1}, x_t] + b_C)$$

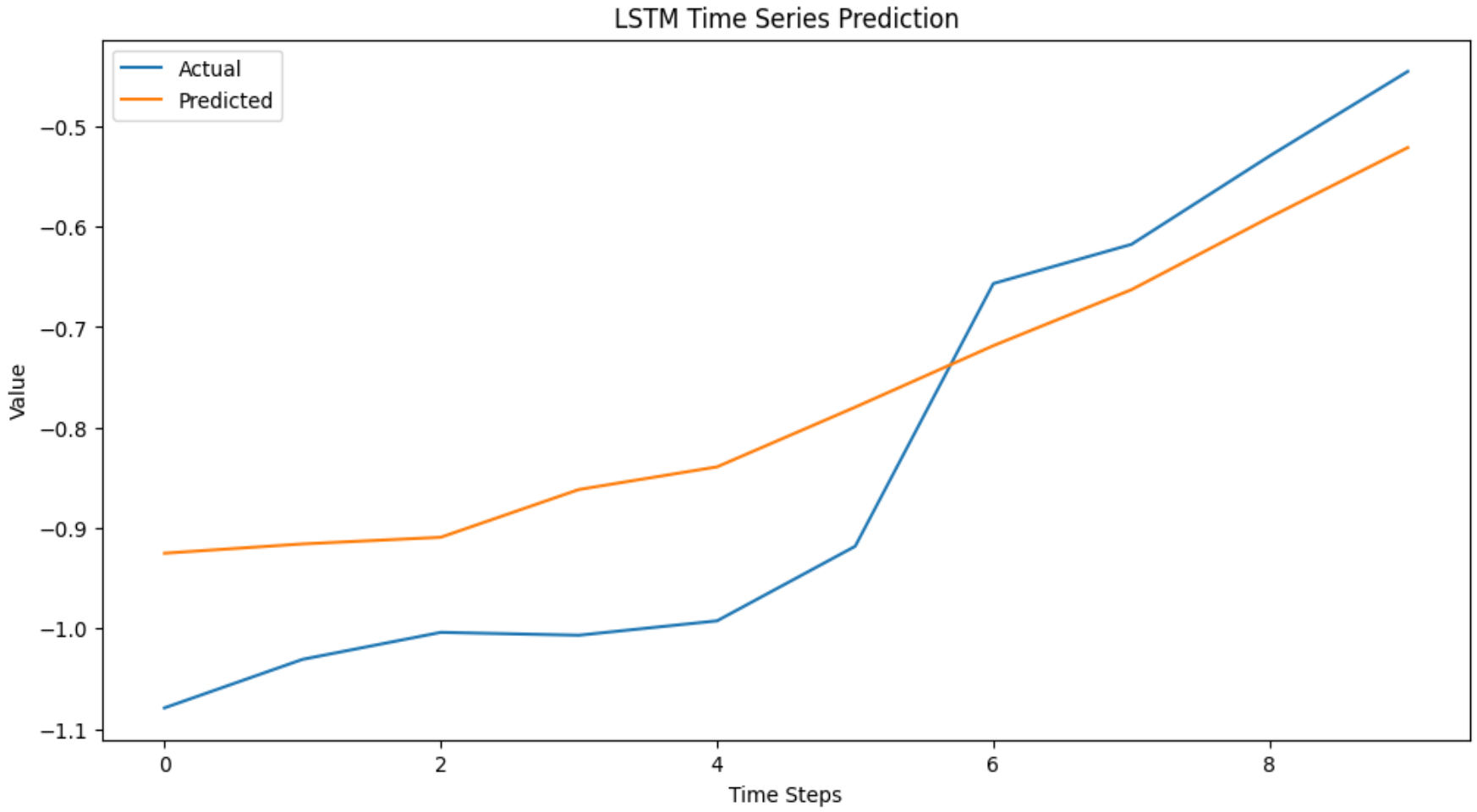
- ***Cell State* Update**: Combines the forget gate and input gate to update the cell state.

$$C_t = f_t . C_{t-1} + i_t . \tilde{c}_t$$

- ***Output Gate***: Determines the output of the current time step based on the updated cell state.

$$o_t = \sigma(W_o . [h_{t-1}, x_t] + b_o)$$

Long Short-Term Memory (LSTM):



Advantages and Limitations of LSTM

Advantages:

- **Handles Long-Term Dependencies**: Effectively captures long-range dependencies in data.
- **Robust Against Vanishing Gradients**: Gating mechanisms prevent gradients from becoming too small.
- **Versatility**: Performs well across diverse tasks like NLP, time series, and anomaly detection.

Limitations:

- **Computational Overhead**: Higher training time compared to basic RNNs.
- **Complexity**: Requires careful tuning of hyperparameters.
- **Data-Intensive**: Performance improves significantly with larger datasets.

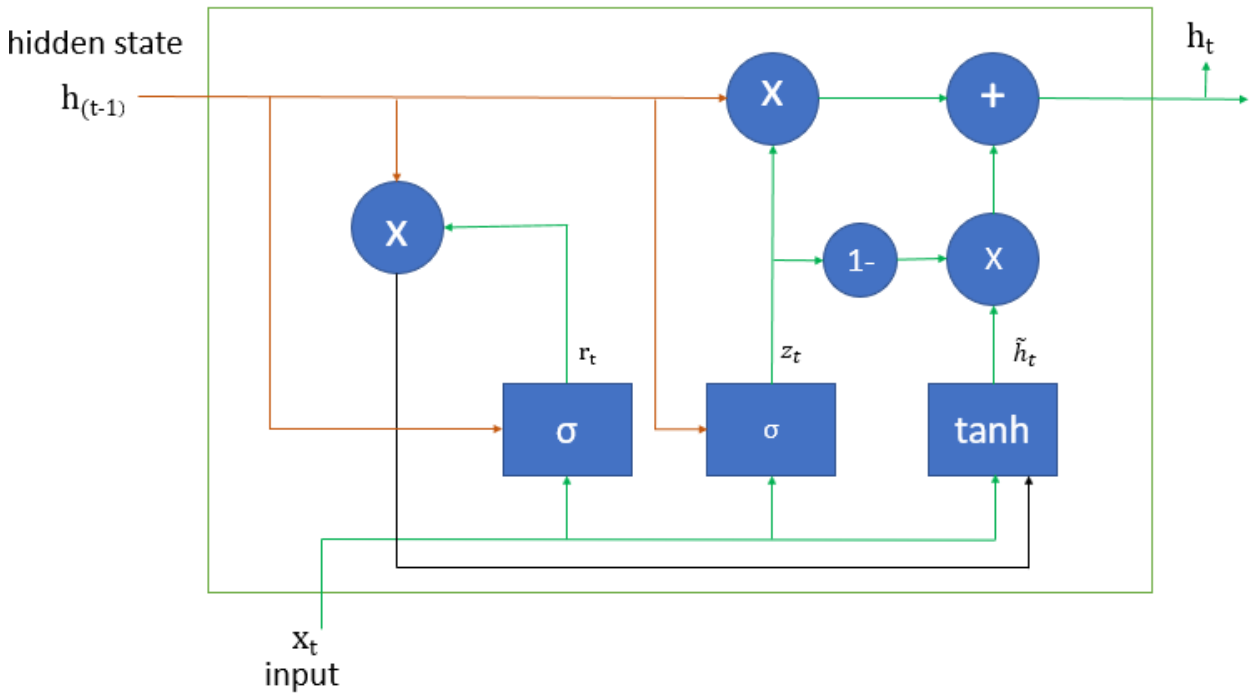
Gated Recurrent Units (GRU):

- GRUs reduce the complexity of LSTMs by combining certain gates while maintaining comparable performance in many tasks.

Key Characteristics:

- **Fewer Parameters**: Simplified structure compared to LSTMs.
- **Efficient Training**: Faster convergence due to fewer computations.
- **Comparable Performance**: Performs well on tasks requiring sequential data analysis.

Gated Recurrent Units (GRU):



Update Gate:

- Determines how much of the previous hidden state (h_{t-1}) to carry forward and how much new information from the current input (x_t) to include.
- Formula:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

- **Purpose**: Controls the balance between retaining past information and incorporating new data.

Gated Recurrent Units (GRU):

Reset Gate:

- Decides how much of the previous hidden state to forget.
- Formula:

$$r_t = \sigma\,(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Purpose: Determines the relevance of past information for computing the new hidden state.

Candidate Hidden State: Represents the potential state of the unit based on the current input and reset gate.

- : Candidate hidden state.

Gated Recurrent Units (GRU):

Hidden State Update:

- Combines the gates to compute the current hidden state (h_t)
- Formula:

$$\hat{h} = tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \,.\, h_{t-1} + z_t \,.\, \hat{h}_t$$

- **Purpose**: Updates the hidden state using the reset gate and update gate, blending past and new information.

Gated Recurrent Units (GRU):

Advantages:

- **Simpler Architecture**: Fewer parameters than LSTM, leading to faster training.
- **Efficient for Small Datasets**: Performs well even with limited data.
- **Effective for Long-Term Dependencies**: Handles sequential data effectively.

Limitations:

- **Limited Expressiveness**: May underperform on tasks requiring very complex dependencies compared to LSTM.
- **Lacks Separate Memory**: Does not have a dedicated cell state like LSTM.
- **Performance Variability**: Effectiveness may vary depending on the dataset.

Generative Adversarial Networks (GANs)

- consist of two neural networks: a **generator** and a **discriminator**.

- These networks operate in a competitive framework, where they engage in a dynamic interplay to produce highly realistic synthetic data.

- The generator is trained to minimize the loss function that measures how well it can deceive the discriminator, while the discriminator is trained to maximize its ability to correctly classify real and fake data.

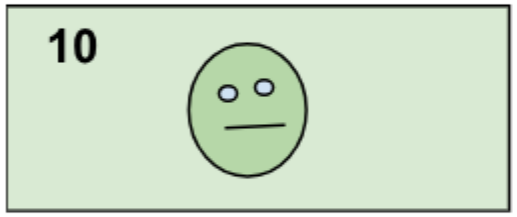
Generator (G): Learns to create data that mimics the real dataset.
Discriminator (D): Learns to distinguish between real and generated data.

Generative Adversarial Networks (GANs)

Generated Data **Discriminator** **Real Data**

FAKE REAL

As training progresses, the generator gets closer to producing output that can fool the discriminator:



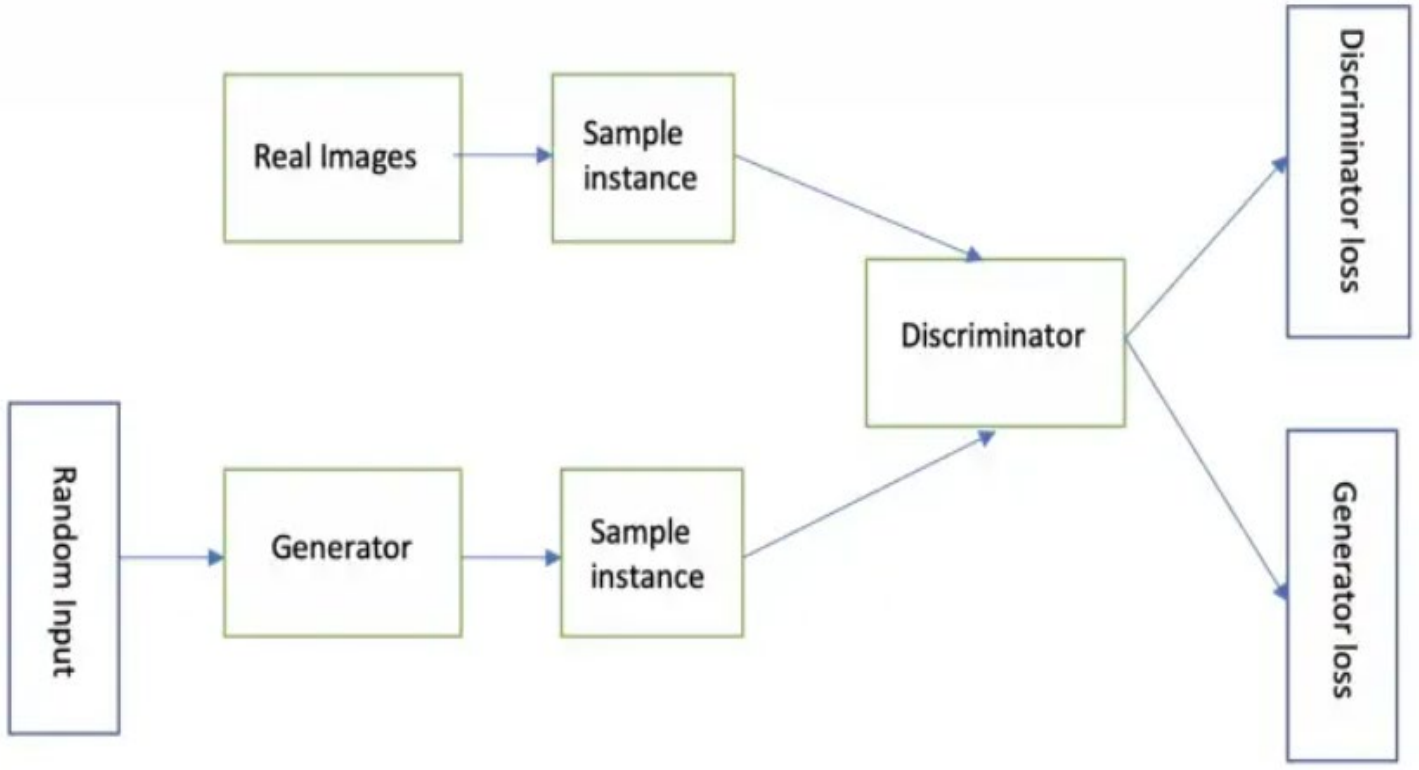
FAKE REAL

Finally, if generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.

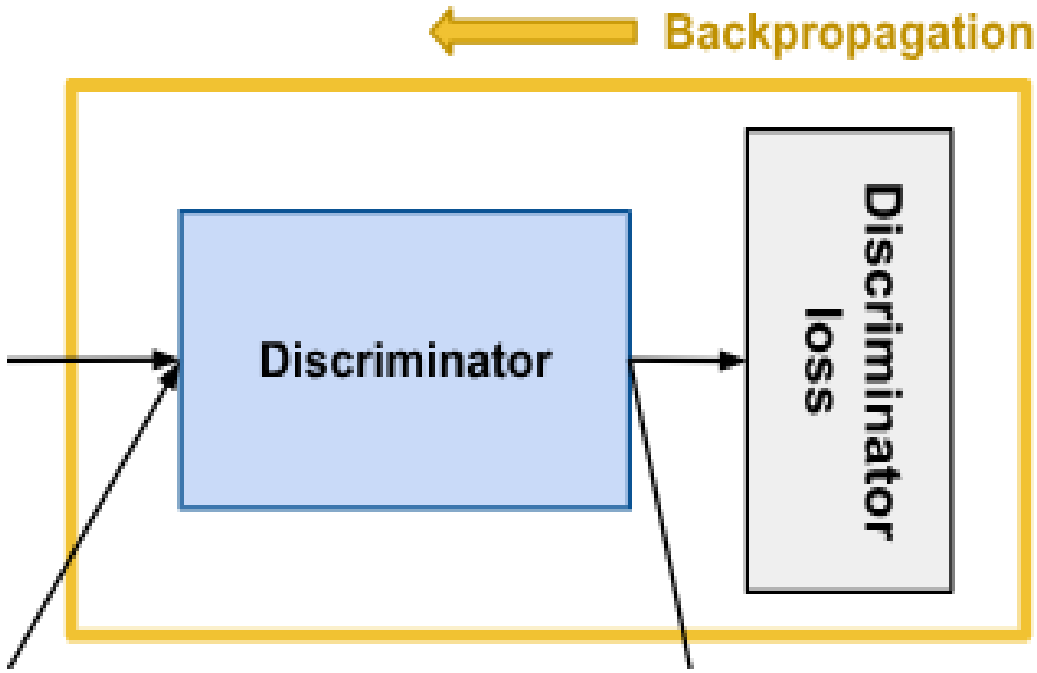
REAL REAL

https://developers.google.com/machine-learning/gan/gan_structure

Generative Adversarial Networks (GANs)



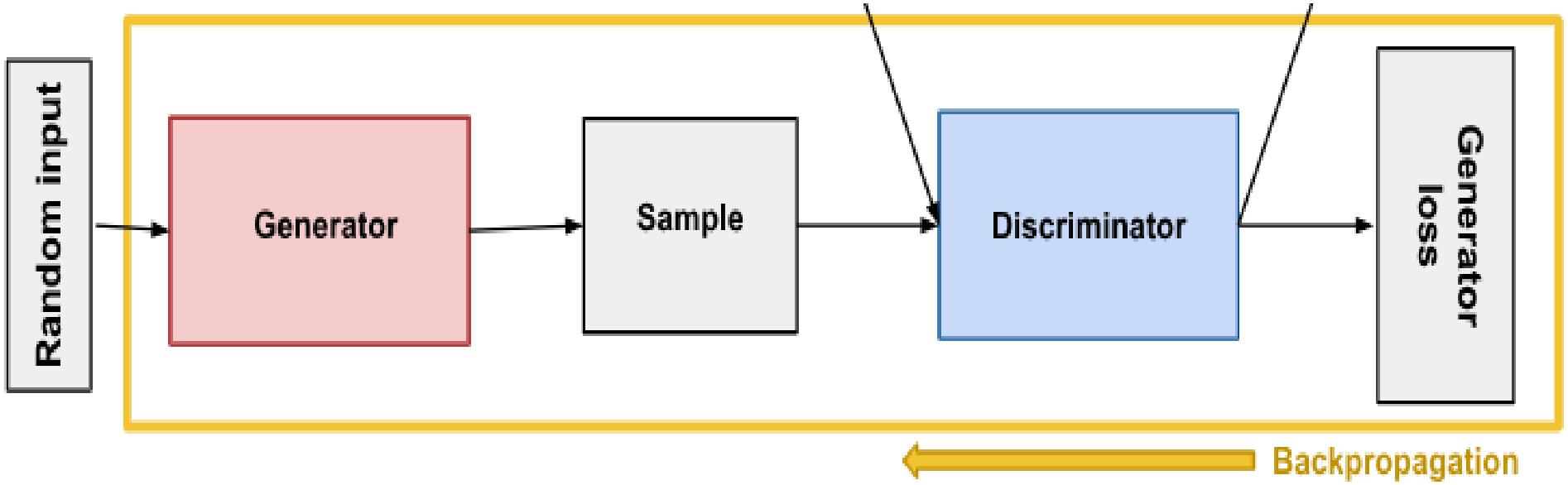
The Discriminator:



There are two sources of training data for the discriminator:

- Real data examples, like actual human photos. These incidents serve as beneficial training examples for the discriminator.
- Fake data examples that the generator produced. These incidents serve as negative training examples for the discriminator.

The Generator:



A GAN's generator component uses the discriminator's feedback to learn how to produce fictitious data. It gains the ability to convince the discriminator that its output is real.

Homework 2

1) Proceed on your Homework 1
2) Add mode ANN models to the bottom of your current homework
3) ANN models would be LSTM, GRU
4) You are free to choose any hyper-parameter values, number of hidden layers, number of perceptront at each level and activation functions
5) Comment on your findings
6) Due 10May2026